# SWEN30006 Software Modelling and Design
# Workshop 5: Fundamental Design Patterns (2) – Partial Solution

School of Computing and Information Systems
University of Melbourne
Semester 1, 2025

## Exercise 1

By this stage you should be getting your UML diagram notation correct. Some relevant sections of the textbook: Chapter 9 Domain Models – especially 9.5 Guideline: How to find Conceptual Classes; and 9.14 Associations

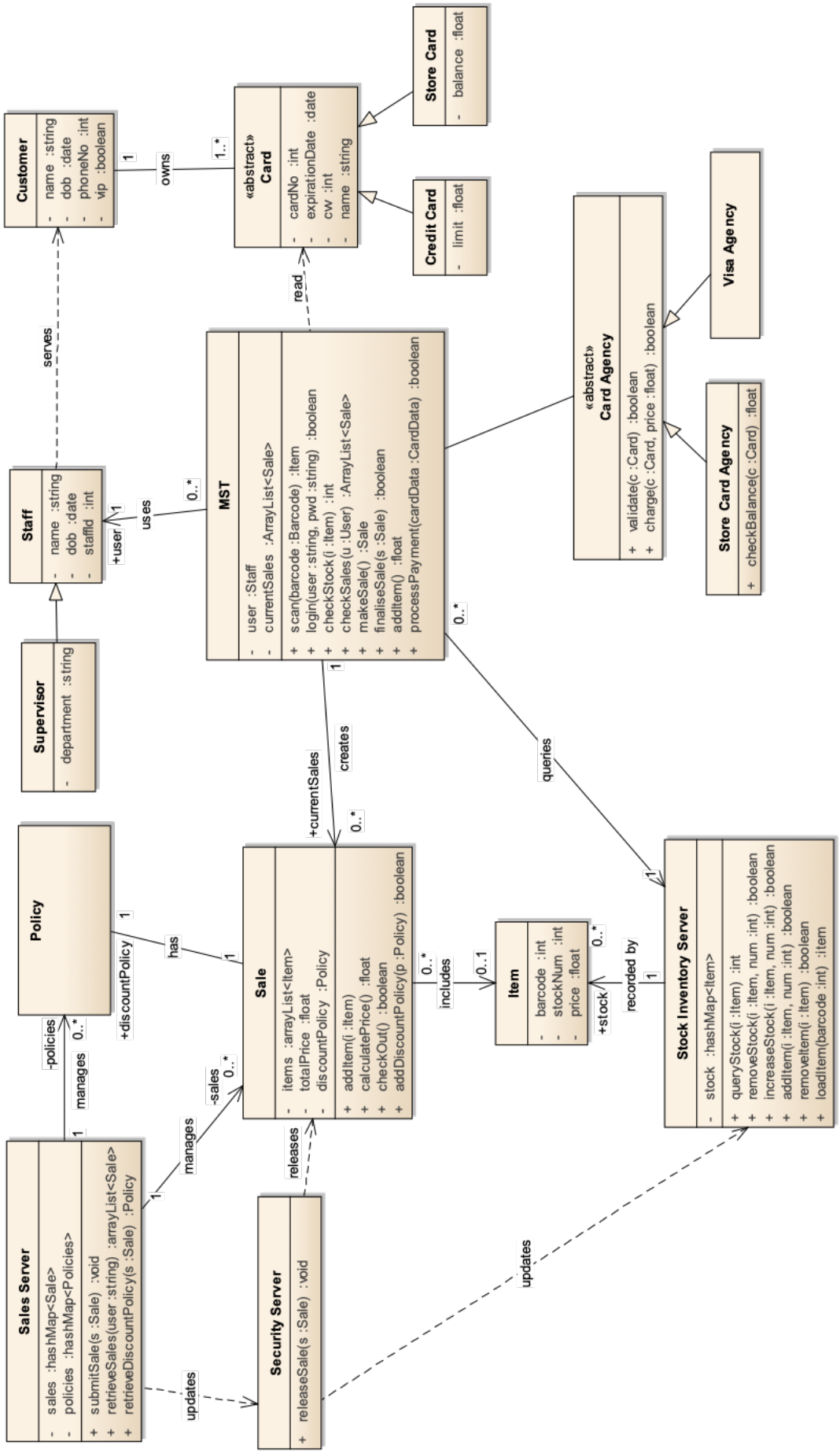
## Exercise 2

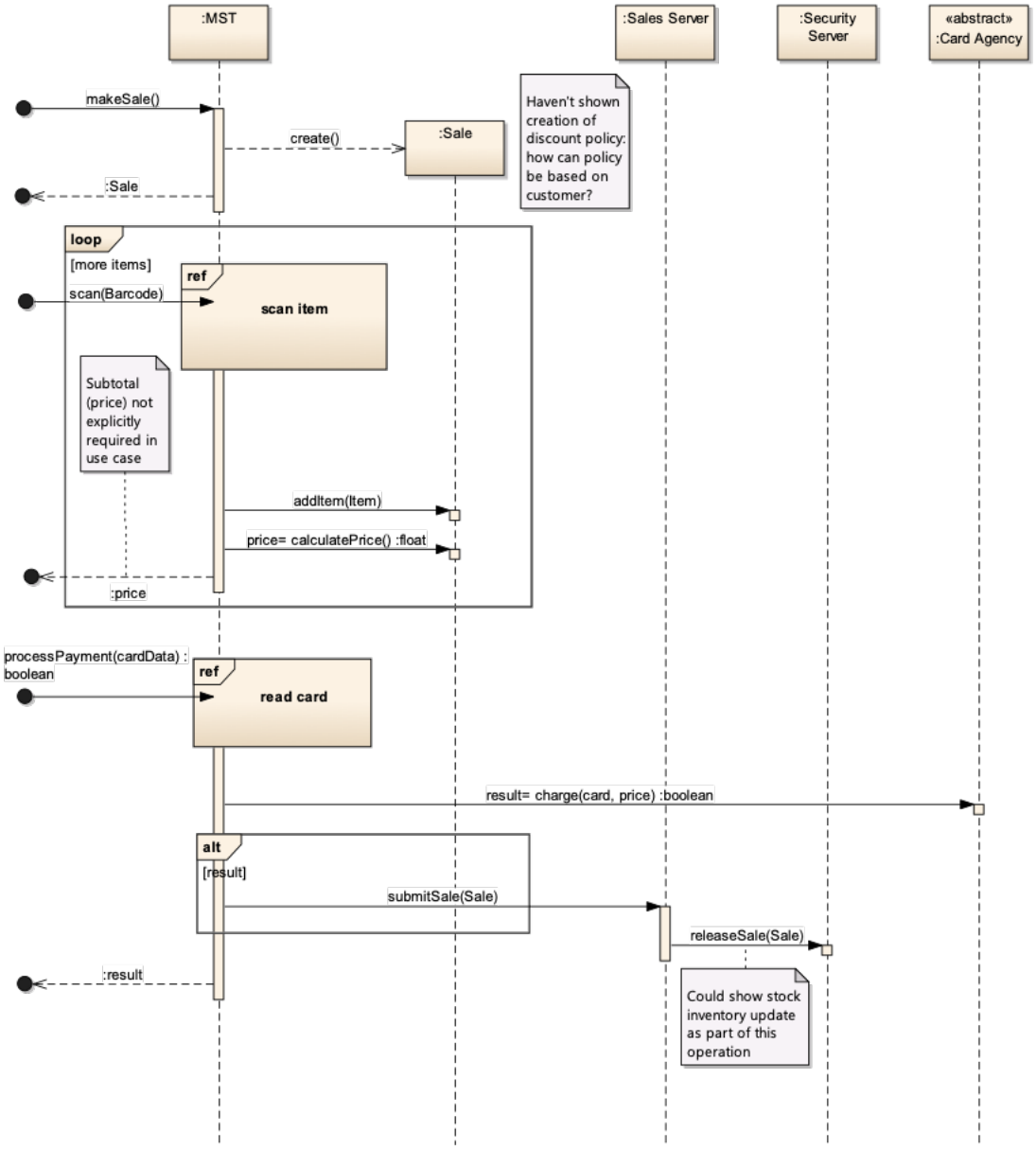
This is like a follow on from Workshop 4. Some relevant sections of the textbook:

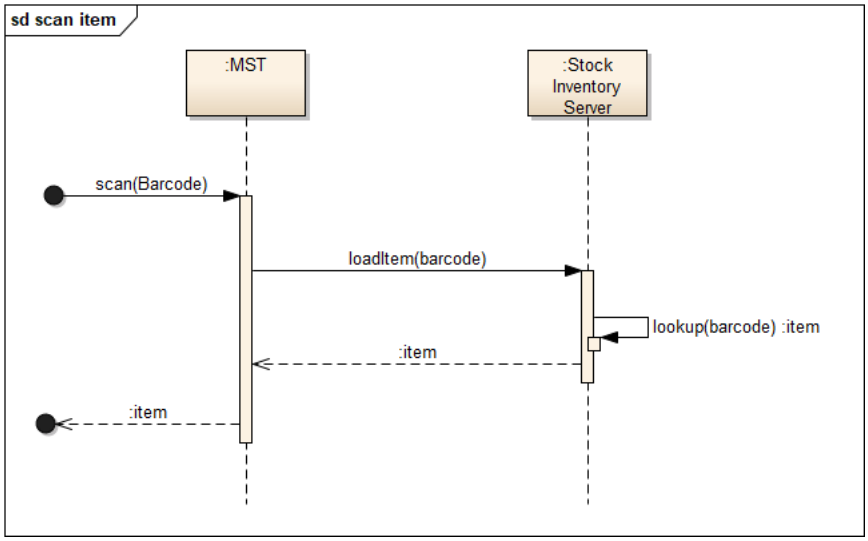
- Chapter 17 GRASP: Designing Objects with Responsibilities (esp.)
- Chapter 19 Designing for Visibility
- Chapter 25 GRASP: More Objects with Responsibilities

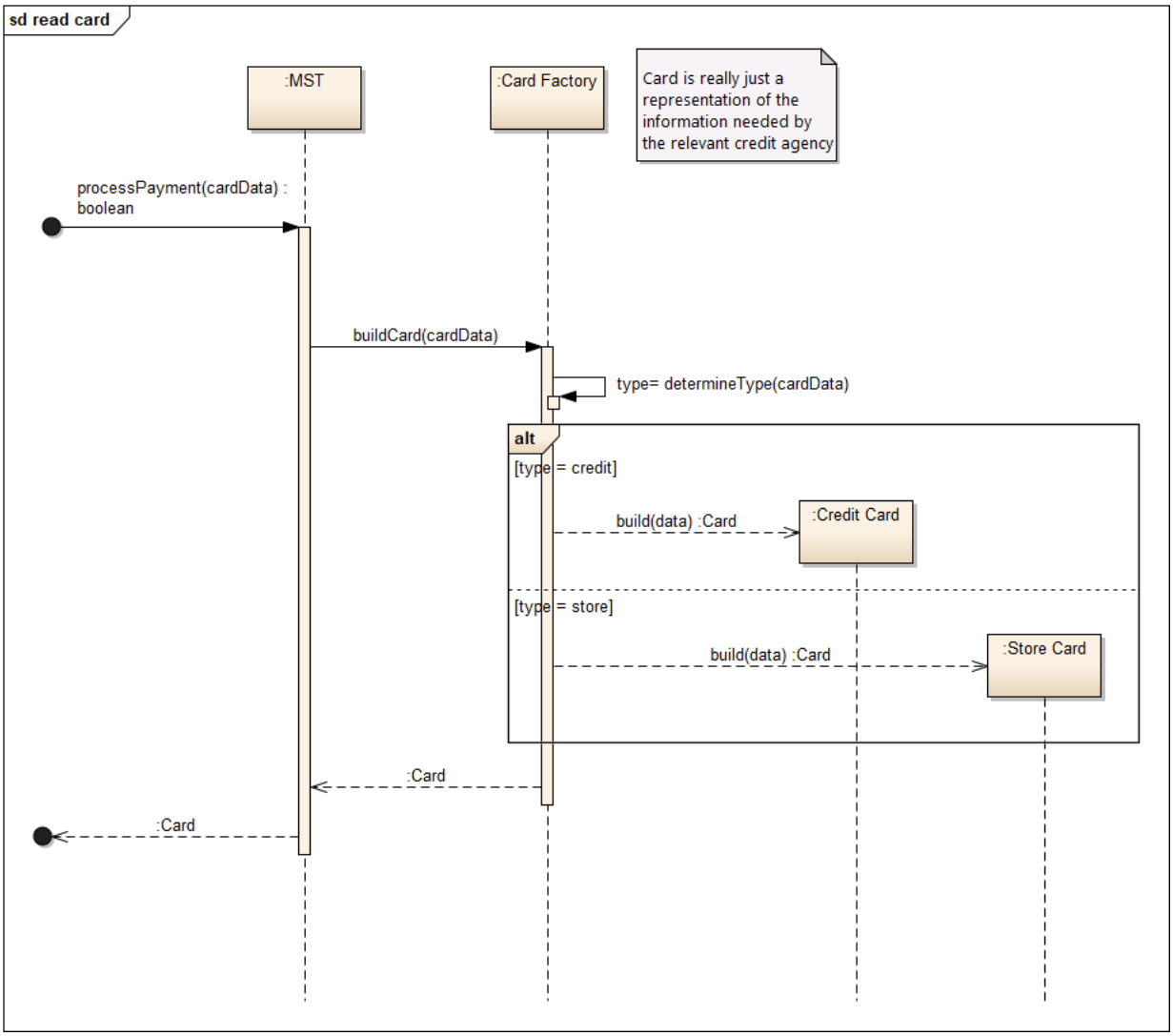
Your design model will vary according to your interpretation of the requirements and the domain model you produce, and you should refine your design model after dynamic modelling. The diagram that follows is an indicative result after that refinement.

Example sequence diagram: note that your sequence diagram will likely look different given your domain models and static design model.


Customer
- name :string
- dob :date
- phoneNo :int
- vip :boolean
owns
1
1..*
«abstract»
Card
- cardNo :int
- expirationDate :date
- cw :int
- name :string
Store Card
- balance :float
Credit Card
- limit :float
read
serves
Staff
- name :string
- dob :date
- staffId :int
+user
1
uses
0..*
MST
- user :Staff
- currentSales :ArrayList<Sale>
+ scan(barcode :Barcode) :Item
+ login(user :string, pwd :string) :boolean
+ checkStock(i :Item) :int
+ checkSales(u :User) :ArrayList<Sale>
+ makeSale() :Sale
+ finaliseSale(s :Sale) :boolean
+ addItem() :float
+ processPayment(cardData :CardData) :boolean
«abstract»
Card Agency
+ validate(c :Card) :boolean
+ charge(c :Card, price :float) :boolean
Visa Agency
Store Card Agency
+ checkBalance(c :Card) :float
Supervisor
- department :string
+currentSales
creates
1
0..*
queries
0..*
1
Policy
+discountPolicy
1
has
1
Sale
- items :arrayList<Item>
- totalPrice :float
- discountPolicy :Policy
+ addItem(i :Item)
+ calculatePrice() :float
+ checkOut() :boolean
+ addDiscountPolicy(p :Policy) :boolean
0..*
includes
0..1
Item
- barcode :int
- stockNum :int
- price :float
0..*
+stock
recorded by
1
Stock Inventory Server
- stock :hashMap<Item>
+ queryStock(i :Item) :int
+ removeStock(i :Item, num :int) :boolean
+ increaseStock(i :Item, num :int) :boolean
+ addItem(i :Item, num :int) :boolean
+ removeItem(i :Item) :boolean
+ loadItem(barcode :int) :item
-policies
0..*
manages
1
-sales
0..*
manages
1
Sales Server
- sales :hashMap<Sale>
- policies :hashMap<Policies>
+ submitSale(s :Sale) :void
+ retrieveSales(user :string) :arrayList<Sale>
+ retrieveDiscountPolicy(s :Sale) :Policy
updates
releases
Security Server
+ releaseSale(s :Sale) :void
updates


:MST
:Sales Server
:Security Server
«abstract» :Card Agency
makeSale()
create()
:Sale
Haven't shown creation of discount policy: how can policy be based on customer?
:Sale
loop
[more items]
ref
scan(Barcode)
scan item
Subtotal (price) not explicitly required in use case
addItem(Item)
price= calculatePrice() :float
:price
processPayment(cardData) : boolean
ref
read card
result= charge(card, price) :boolean
alt
[result]
submitSale(Sale)
releaseSale(Sale)
:result
Could show stock inventory update as part of this operation


sd scan item
:MST
:Stock Inventory Server
scan(Barcode)
loadItem(barcode)
lookup(barcode) :item
:item
:item


sd read card
:MST
:Card Factory
Card is really just a representation of the information needed by the relevant credit agency
processPayment(cardData) : boolean
buildCard(cardData)
type= determineType(cardData)
alt
[type = credit]
build(data) :Card
:Credit Card
[type = store]
build(data) :Card
:Store Card
:Card
:Card